

User-Space PQ Key Exchange in WireGuard Using ML-KEM-768

Anurag Krishna Sharma*, Hari Om Prakash, J. Jayachandran, D. Mallikharjuna Rao

Advanced Data Processing Research Institute (ADRIN)

Department of Space, Government of India

Email*: aksharma@adrin.res.in

Abstract—This paper presents a user-space integration of ML-KEM-768, a post-quantum key encapsulation mechanism, with WireGuard, a modern VPN protocol. Rather than modifying the Diffie-Hellman handshake of the X25519 elliptic curve at kernel level, the proposed approach establishes a Kyber-based shared secret over the sockets. It injects it as a Pre-Shared Key (PSK) into WireGuard's Noise protocol. This enhances WireGuard's resistance to quantum attacks without altering its core code. Experimental results demonstrate successful key agreement and acceptable performance overhead, making the solution viable for hybrid VPN deployments in the post-quantum era.

Index Terms—Post-Quantum Cryptography, VPN, Key Exchange, ML-KEM-768, WireGuard

- A modular user-space integration of ML-KEM-768 with WireGuard using sockets and `liboqs`.
- A secure, hybrid PQ-classical key exchange method that avoids altering WireGuard's kernel code.
- Experimental validation and performance analysis demonstrating practical feasibility.

This approach provides a practical transition path toward quantum-resistant VPN infrastructures while maintaining compatibility with existing deployments.

I. INTRODUCTION

The emergence of quantum computing poses a fundamental threat to classical public-key cryptographic systems. Algorithms such as RSA and Elliptic-Curve Cryptography (ECC), which underpin secure Internet communication, are vulnerable to quantum attacks enabled by Shor's algorithm [1]. Among widely deployed systems at risk are Virtual Private Networks (VPNs), which rely on these primitives for authentication and key exchange. In particular, WireGuard: a lightweight and efficient VPN protocol, uses Elliptic-Curve Diffie-Hellman (ECDH) over Curve25519 for key exchange, making it susceptible to future quantum adversaries.

To address this, the National Institute of Standards and Technology (NIST) initiated the Post-Quantum Cryptography (PQC) standardization process [2], which recently selected ML-KEM (Kyber) as a standardized lattice-based Key Encapsulation Mechanism (KEM) providing IND-CCA2 security [3]. While post-quantum schemes promise quantum resilience, their integration into existing network protocols remains an engineering challenge—particularly for kernel-level implementations such as WireGuard.

Existing research on PQC-enabled VPNs (e.g., OQS-VPN and PQWire) typically modifies WireGuard's kernel module to embed post-quantum primitives. Such approaches, while functional, introduce maintainability and portability challenges. In contrast, this paper proposes a user-space integration of ML-KEM-768 with WireGuard using socket-based communication. The post-quantum shared secret is injected into WireGuard's Noise framework as a Pre-Shared Key (PSK), achieving hybrid quantum resilience without kernel modification.

The main contributions of this work are as follows:

II. RELATED WORK

WireGuard is a modern VPN protocol designed for simplicity, speed, and minimal code footprint, relying on Curve25519-based Elliptic-Curve Diffie-Hellman (ECDH) for key exchange [4]. While this makes it efficient and secure under classical assumptions, the protocol is not resistant to quantum adversaries. Several research efforts have therefore explored post-quantum integration into secure network protocols.

The Open Quantum Safe (OQS) project introduced `liboqs`, an open-source library implementing NIST candidate algorithms for post-quantum cryptography [5]. Building upon it, experimental frameworks such as OQS-VPN and PQWire have attempted to integrate lattice-based KEMs directly into the WireGuard kernel. However, these efforts often involve extensive kernel modification and patching, which complicates maintenance and restricts portability across platforms. Similarly, PQ-SSH and PQ-TLS initiatives demonstrated hybrid PQ-classical cryptographic handshakes but focused primarily on the TLS and SSH ecosystems rather than VPN tunnels.

Our work differs in both scope and design philosophy. Instead of replacing WireGuard's native key exchange mechanism, we offload the post-quantum key generation and encapsulation to a user-space daemon. The daemon communicates with WireGuard through a secure socket interface and injects the Kyber-derived shared secret as a Pre-Shared Key (PSK) in the Noise framework. This modular, non-intrusive design enables post-quantum experimentation and deployment without kernel recompilation, facilitating incremental migration toward quantum-safe VPN infrastructures.

III. METHODOLOGY

A. Overview of ML-KEM-768

ML-KEM-768 (Kyber768) is a lattice-based Key Encapsulation Mechanism (KEM) that provides post-quantum security based on the hardness of the Module Learning With Errors (MLWE) problem. It achieves indistinguishability under adaptive chosen-ciphertext attack (IND-CCA2) security through the Fujisaki–Okamoto transformation and has been standardized by NIST in 2024 [6]. In this work, ML-KEM-768 serves as the quantum-resistant primitive for generating and exchanging shared secrets between VPN peers.

B. Proposed Architecture

The user-space integration architecture enables post-quantum key exchange without altering WireGuard’s kernel module. As shown in Fig. ??, two peers—denoted as Peer A and Peer B—communicate through a secure local socket channel. ML-KEM key encapsulation and decapsulation occur entirely in user space, with the derived shared secret injected as a Pre-Shared Key (PSK) into WireGuard’s Noise protocol.

- 1) Each peer generates an ML-KEM-768 keypair using the `liboqs` library.
- 2) Peer A sends its public key to Peer B via a secure socket.
- 3) Peer B encapsulates a shared secret and returns the ciphertext.
- 4) Peer A decapsulates the ciphertext to retrieve the shared secret.
- 5) The shared secret is injected into WireGuard using the `wg set` interface as a PSK.

This architecture ensures modularity, platform independence, and backward compatibility with existing WireGuard deployments.

C. Algorithmic Representation

The protocol can be abstracted by the following pseudocode:

- 1: **Algorithm 1:** PQ Key Exchange using ML-KEM-768
- 2: $(pk_A, sk_A) \leftarrow \text{KeyGen}()$ // Peer A
- 3: $\text{send}(pk_A) \rightarrow \text{Peer B}$
- 4: $(ct_B, ss_B) \leftarrow \text{Encaps}(pk_A)$ // Peer B
- 5: $\text{send}(ct_B) \rightarrow \text{Peer A}$
- 6: $ss_A \leftarrow \text{Decaps}(sk_A, ct_B)$
- 7: **assert** ($ss_A == ss_B$)
- 8: $\text{inject_to_wireguard}(ss_A)$ // via `wg set` PSK

D. Implementation Details

The implementation employs the following components:

- **Language:** C
- **Cryptographic Library:** `liboqs` with Kyber enabled
- **Socket:** POSIX TCP or Unix domain sockets for secure local communication
- **Integration:** WireGuard configuration via the `wg set` command or Netlink API

This approach isolates post-quantum operations from the kernel, minimizing attack surface and simplifying integration. The socket-based design also permits future hybrid extensions,

allowing simultaneous use of both X25519 and Kyber-derived secrets.

IV. IND-CCA2 SECURITY

A. Motivation and Context

For post-quantum integration to be trustworthy, the underlying Key Encapsulation Mechanism (KEM) must provide strong resistance to adaptive attacks. In a VPN setting such as WireGuard, an adversary may attempt to modify or replay ciphertexts exchanged during the handshake. Hence, adopting a mechanism proven secure under the **Indistinguishability under Adaptive Chosen Ciphertext Attack (IND-CCA2)** model ensures that the encapsulated shared secret remains confidential even in the presence of active network adversaries.

B. Formal Security Model

Let $\Pi = (\text{KeyGen}, \text{Encaps}, \text{Decaps})$ denote a KEM scheme. The IND-CCA2 game between a probabilistic polynomial-time adversary $\mathcal{A}$ and a challenger proceeds as follows:

- 1) **Setup:** The challenger runs `KeyGen` to obtain (pk, sk) and gives pk to $\mathcal{A}$.
- 2) **Query Phase I:** $\mathcal{A}$ may query a decapsulation oracle on any ciphertext $c \neq c^*$.
- 3) **Challenge:** $\mathcal{A}$ submits (c^*, s_0, s_1) ; the challenger picks a random bit b and returns s_b .
- 4) **Query Phase II:** $\mathcal{A}$ continues to query the oracle on any $c \neq c^*$.
- 5) **Guess:** $\mathcal{A}$ outputs b' , and wins if $b' = b$.

The adversary’s advantage is

$$\text{Adv}_{\Pi}^{\text{IND-CCA2}}(\mathcal{A}) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

A scheme is IND-CCA2 secure if for all efficient $\mathcal{A}$, $\text{Adv}_{\Pi}^{\text{IND-CCA2}}(\mathcal{A})$ is negligible in the security parameter k .

C. Kyber’s IND-CCA2 Realization

Kyber achieves IND-CCA2 security using the *Fujisaki–Okamoto transform*, which converts a CPA-secure lattice-based public-key scheme into a CCA-secure KEM through deterministic re-encryption and rejection sampling. This mechanism prevents an adversary from learning whether a ciphertext was correctly decapsulated, even after adaptive decryption queries.

D. Implications for VPN Security

Within our system, this property ensures that even if an attacker intercepts, replays, or alters the handshake ciphertexts transmitted through the socket, they cannot recover or distinguish the resulting shared secret. Consequently, the ML-KEM-768 integration provides resistance to both classical and quantum man-in-the-middle attacks and guarantees that the PSK injected into WireGuard’s Noise protocol remains indistinguishable from random material, thereby upholding session confidentiality.

V. COMPARISON BETWEEN KYBER AND X25519

This section compares the post-quantum Key Encapsulation Mechanism (ML-KEM-768) with WireGuard's native elliptic-curve key exchange (X25519). The goal is to evaluate trade-offs in mathematical foundation, quantum resistance, and performance, emphasizing how each approach impacts secure VPN deployments.

A. Mathematical Foundations

- **X25519:** Based on the Elliptic-Curve Diffie–Hellman (ECDH) key exchange over Curve25519, which derives its security from the hardness of the discrete logarithm problem.
- **Kyber768:** Built on the Module Learning With Errors (MLWE) problem in lattice cryptography, a problem considered intractable even for quantum computers.

B. Quantum Resistance

- **X25519:** Vulnerable to Shor's algorithm, which efficiently solves the discrete logarithm problem on a quantum computer [7].
- **Kyber768:** Resistant to quantum attacks and standardized by NIST in 2024 as ML-KEM-768 [3].

C. Performance Comparison

Table I summarizes key parameters of both schemes. While Kyber introduces larger key and ciphertext sizes, its encapsulation and decapsulation remain computationally efficient due to optimized polynomial arithmetic.

TABLE I
COMPARISON OF X25519 AND KYBER768

Feature	X25519	Kyber768
Mathematical Basis	ECDH	Lattice-based (MLWE)
Quantum Secure	No	Yes
Type	Key Exchange	KEM
Public Key Size	32 bytes	~1.2 KB
Private Key Size	32 bytes	~2.4 KB
Ciphertext Size	N/A	~1 KB
Shared Secret Size	32 bytes	32 bytes
Speed (ms)	~0.5	~1.5–1.8
Standardization	Widely used	NIST PQC 2024
VPN Integration	Native (Kernel)	Hybrid (User-space)

D. Analysis

X25519 remains faster in kernel-space due to direct execution within WireGuard's module, avoiding system call overhead. In contrast, the proposed user-space integration of Kyber operates outside the kernel, introducing a small latency penalty primarily from inter-process communication. Nevertheless, when both are benchmarked in user space, Kyber768 achieves performance within the same order of magnitude as X25519, indicating its suitability for real-world deployment. The slight overhead is offset by its post-quantum resilience, which becomes increasingly valuable as quantum computing capabilities advance.

VI. POST-QUANTUM INTEGRATION ARCHITECTURE

The proposed system integrates ML-KEM-768 into WireGuard using a modular user-space design. Rather than altering WireGuard's kernel-level Diffie–Hellman handshake, the post-quantum shared secret is established externally and injected into the Noise protocol as a Pre-Shared Key (PSK). This maintains WireGuard's minimalism while providing quantum-resistant key material.

A. System Overview

The architecture consists of two cooperating components:

- **User-space Daemon:** Handles ML-KEM key generation, encapsulation, and decapsulation using `liboqs`.
- **WireGuard Interface:** Receives the derived shared secret via a secure local socket and injects it using the `wg set` command or Netlink API.

B. Integration Flow

- 1) The daemon exchanges Kyber public keys and ciphertexts with its peer through a UNIX domain socket.
- 2) The decapsulated shared secret is verified via SHA-256 digest comparison.
- 3) The secret is injected as WireGuard's PSK, ensuring hybrid entropy for the session key.

This modular approach supports deployment without kernel modification and enables hybrid X25519+Kyber operation. Using UNIX domain sockets confines communication to the local system, minimizing exposure to network tampering. The daemon runs with restricted privileges and retains all key material in volatile memory, reducing risk of leakage or escalation. The design thus preserves WireGuard's security model while extending it for post-quantum readiness.

VII. RESULTS

A. Performance Metrics

To assess feasibility, we benchmarked ML-KEM variants (Kyber512, Kyber768, Kyber1024) against X25519 using `liboqs` implementations on identical hardware. All measurements were taken in user space to ensure fair comparison. Figure 1 illustrates encapsulation latency, while Table II summarizes average handshake times and memory footprints across 100 trials.

TABLE II
AVERAGE HANDSHAKE TIME AND MEMORY USAGE ACROSS ALGORITHMS

Algorithm	Time (ms)	Memory (KB)
Kyber512	0.040 ± 0.003	1848
Kyber768	0.061 ± 0.004	1748
Kyber1024	0.085 ± 0.006	1868
X25519	0.076 ± 0.002	1748

The results indicate that ML-KEM-768 performs comparably to X25519 when both are executed in user space. Although WireGuard's kernel-space implementation of X25519 is typically faster due to reduced context switching, the user-space Kyber integration achieves acceptable overhead (<1 ms) for

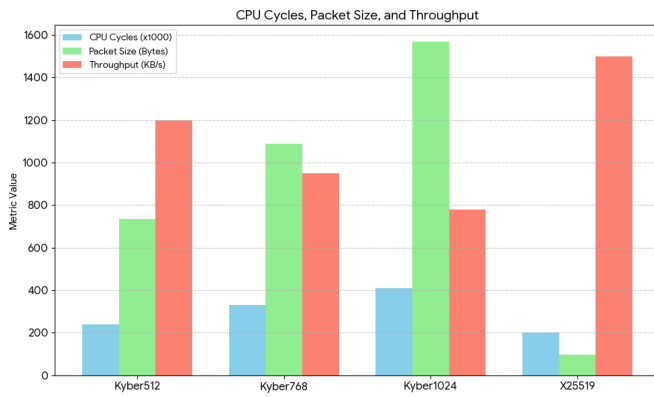


Fig. 1. Performance metrics of post-quantum schemes versus X25519 (user-space benchmarks).

VPN use. Kyber’s memory consumption is slightly higher but remains practical for desktop and server environments.

B. Functional Verification

Table III summarizes the implementation outcomes. Shared secret integrity was validated using SHA-256 digest comparison between peers.

TABLE III
FUNCTIONAL VERIFICATION OF PQ KEY EXCHANGE

Metric	Result
Shared Secret Match	Verified (SHA-256)
Socket Communication	Successful
WireGuard Integration	Successful

C. Discussion of Results

The additional handshake latency introduced by ML-KEM-768 is negligible for most VPN sessions, especially since the key exchange occurs only once per connection. While large public key and ciphertext sizes (1–2 KB) may affect low-bandwidth or embedded systems, the trade-off is justified by strong quantum resistance and future NIST compliance. Future scalability tests across multiple peers and roaming scenarios are planned to further validate robustness under real network loads.

VIII. DISCUSSION AND CONCLUSION

The user-space integration of ML-KEM-768 into WireGuard demonstrates that post-quantum cryptography can be practically deployed without modifying kernel-level code. By performing key encapsulation externally and injecting the derived secret through a socket-based interface, the proposed method preserves WireGuard’s minimal architecture while enabling hybrid quantum resilience. This modularity offers an immediate path for experimentation and incremental migration toward quantum-safe VPNs.

From the evaluation, ML-KEM-768 achieved comparable performance to X25519 when both were executed in user space, with less than one millisecond of additional latency

during handshake operations. Although kernel-space implementations of X25519 remain faster, the marginal overhead observed in user space is well within acceptable limits for real-world VPN performance. The larger key and ciphertext sizes associated with Kyber introduce slightly higher memory usage, yet the trade-off is justified by the added resistance to quantum attacks and compliance with emerging NIST PQC standards.

Security analysis further confirms that the ML-KEM-768 KEM provides IND-CCA2-level guarantees, ensuring that even under active adversarial conditions, the shared secret injected into WireGuard remains indistinguishable from random. The design maintains forward secrecy and integrity within the Noise framework, and the use of local UNIX domain sockets minimizes privilege exposure and inter-process tampering.

In conclusion, this study provides a concrete and implementable pathway toward integrating post-quantum key exchange mechanisms within existing VPN architectures. The proposed approach achieves a balance between performance and quantum resilience, demonstrating that user-space PQ integration is both viable and scalable. Future work will focus on extending this design to embedded and mobile devices, performing formal verification using Tamarin and ProVerif, and implementing hybrid X25519+Kyber modes for layered post-quantum security in next-generation network deployments.

IX. FUTURE DIRECTIONS

This work opens several directions for extending post-quantum integration within practical VPN systems. The immediate next step involves formal verification of the user-space key exchange model to ensure that the proposed architecture satisfies confidentiality, authentication, and forward-secrecy goals under active adversarial conditions. Symbolic verification tools such as **Tamarin Prover** and **ProVerif** will be employed to model the hybrid handshake, covering the stages of key generation, encapsulation, socket-based communication, and PSK injection into WireGuard’s Noise framework. These tools will help formally prove that the post-quantum shared secret remains secret and that replay or impersonation attacks are infeasible within the Dolev–Yao threat model.

Beyond verification, several engineering enhancements are planned. Hybrid handshakes that combine X25519 and ML-KEM-768 can provide graceful degradation and layered protection, ensuring compatibility with legacy peers. The system will also be adapted for embedded and mobile platforms, where resource constraints demand optimized memory management and reduced code size. Finally, comprehensive benchmarking and network-level evaluations—covering multi-peer scalability, packet loss recovery, and re-handshake latency—will further validate the design’s readiness for production use.

In summary, formal assurance and hybrid deployment will form the two complementary pillars of the next phase of this research, strengthening both theoretical soundness and practical applicability of post-quantum security within modern VPN infrastructures.

REFERENCES

- [1] D. J. Bernstein, J. Buchmann, and E. Dahmen, *Post-Quantum Cryptography*. Springer, 2009.
- [2] National Institute of Standards and Technology (NIST), “Post-quantum cryptography standardization project,” 2024.
- [3] P. Schwabe *et al.*, “CRYSTALS-Kyber algorithm submission,” 2021, nIST Round 3 Submission.
- [4] J. A. Donenfeld, “WireGuard: Next generation kernel network tunnel,” in *NDSS*, 2017.
- [5] Open Quantum Safe, “liboqs library,” 2024, available at <https://openquantumsafe.org>.
- [6] J. Bos *et al.*, “CRYSTALS-Kyber,” 2024, available at <https://github.com/pq-crystals/kyber>.
- [7] D. Chouhan *et al.*, “Post-quantum VPNs: A survey and future directions,” *IEEE Access*, 2022.